

Aula: Introdução ao JSX no React

🌟 Objetivo da Aula

Entender o que é o **JSX**, por que ele é usado no React, e como utilizá-lo corretamente para criar componentes de forma declarativa, legível e moderna.

🎓 O que é JSX?

JSX significa **JavaScript XML**. É uma extensão de sintaxe para JavaScript que permite escrever **elementos HTML dentro do código JavaScript** de forma muito parecida com HTML tradicional.

```
Unset
function MeuComponente() {
  return <h1>Olá, mundo!</h1>;
}
```

Esse `return <h1>...</h1>` é JSX. No fundo, o React **compila esse JSX em chamadas JavaScript** usando `React.createElement()`.

🔍 Exemplo convertido (JSX → JavaScript puro):

```
Unset
<h1>Olá</h1>

// vira:

React.createElement('h1', null, 'Olá')
```

A função `React.createElement()` tem a seguinte estrutura:

React.createElement(tipo, propriedades, ...filhos)

tipo → 'h1': é o tipo do elemento que será criado.

propriedades → null: seriam os atributos como className, id, style, etc.

filhos → 'Olá': é o conteúdo que aparece dentro da tag (neste caso, o texto "Olá").



Por que usar JSX?

-  Código mais **declarativo**
-  Aproximação do HTML que já conhecemos
-  Mais legível e fácil de manter
-  Integrado com variáveis e lógicas JavaScript

JSX melhora a **experiência de desenvolvimento** sem comprometer a performance, pois é transformado em código JavaScript puro pelo compilador (ex: Vite + Babel ou SWC).



Como escrever JSX corretamente

1. Todo componente retorna um único elemento

Unset

```
// ERRADO
```

```
return (
```

```
  <h1>Oi</h1>
```

```
  <p>Texto</p>
```

```
);
```

```
// CERTO (usando uma tag pai ou fragmento)
```

```
return (
```

```
  <>
```

```
<h1>Oi</h1>

<p>Texto</p>

</>

);
```

2. Atributos em camelCase

Unset

```

```

3. Injeção de valores JS com chaves

Unset

```
const nome = "Maria";

return <p>Olá, {nome}</p>;
```

4. Não pode usar estruturas de controle diretamente (como if)

Unset

```
// ERRADO:

return if (x) { <p>...</p> }

// CERTO:

return x ? <p>Sim</p> : <p>Não</p>;
```

Estrutura de um componente React com JSX (exemplo prático)

App.tsx

Unset

```
export function App() {  
  
  const mensagem = "React com JSX";  
  
  return (  
  
    <main>  
  
      <h1>{mensagem}</h1>  
  
      <button onClick={() => alert("Cliquei!")}>Clique  
aqui</button>  
  
    </main>  
  
  );  
}
```

- Usamos `export function App()` para exportação **nomeada**, o que ajuda na manutenção e identificação de erros.
- JSX permite usar **JS + HTML** de forma integrada.
- O React renderiza apenas o que foi retornado pelo componente.

Hot Reload: como as mudanças aparecem rapidamente

Graças ao Vite e sua arquitetura moderna, toda alteração feita no `App.tsx` é refletida na tela **quase instantaneamente**, sem recarregar a página toda.

Esse comportamento chama-se **Hot Module Replacement (HMR)** ou **Hot Reload**.

! Dicas importantes para JSX

Situação	Solução
Vários elementos sem pai	Use <code><></></code> ou uma <code><div></code>
Atributos com nomes reservados	Use <code>className</code> no lugar de <code>class</code>
Variáveis dentro do JSX	Use <code>{ }</code> para embutir JavaScript
Comentários dentro do JSX	Use <code>{/* assim */}</code>

🚫 O que JSX não é:

- JSX **não é HTML**, embora pareça.
 - JSX **não funciona fora do React** (sem um compilador como Babel ou SWC).
 - JSX **precisa ser convertido** em código JS para funcionar.
-

🌟 Conclusão

JSX é a ponte entre o mundo visual (HTML) e a lógica JavaScript. Ele torna a criação de interfaces mais intuitiva, legível e moderna.

Agora que você domina JSX, está pronto para criar interfaces ricas e reativas com React!

Na próxima aula, vamos explorar como estruturar componentes